AMIGO BOT MATLAB FRAMEWORK

Documentación Técnica de Referencia

Framework de navegación autónoma para el robot AmigoBot sobre ROS y MATLAB

Versión 1.0
25/04/2026

Desarrollado por:
Marcos Escudero Matanza
@MicroxOndas

David Palomeque Santiago
@Paloquete

Tabla de Contenidos

Tabla de Contenidos 2
1. Introducción 4
1.1 Objetivo del documento 4
1.2 Requisitos del sistema 4
2. Arquitectura del Framework 5
2.1 Estructura de ficheros 5
2.2 Diagrama de capas 5
2.3 Flujo de ejecución 6
3. Conexión con ROS 7
3.1 Conexión al simulador (conect.m) 7
3.2 Conexión al robot físico (conect_irl.m) 7
3.3 Topics ROS utilizados 8
3.4 Estructura ros_handles 8
4. MissionControl — API de Alto Nivel 9
4.1 Constructor 9
4.2 Referencia completa de métodos públicos 9
mc.gotoPoint(target, ...) 9
mc.drive(target_local, ...) 9
mc.followWall(side, desired_dist, ...) 10
mc.addWaypoint(wp) y mc.runMission() 10
mc.stop() 10
mc.setVelocity(v, w) 10
mc.setGains(nombre, [Kp Ki Kd]) 10
mc.resetMissionLog() y mc.getMissionLog() 11
5. Configuración de Parámetros 12
5.1 Parámetros de gotoPoint 12
5.2 Parámetros de wallFollow 12
5.3 Parámetros de obstacleAvoid 13
5.4 Modificación directa de Config 13
6. Comportamientos de Navegación 14
6.1 gotoPoint — Modo point-shoot 14
6.2 driveToPoint — Modo continuous 14
6.3 wallFollower — Seguimiento de pared 14
6.4 obstacleAvoidance — Campo de Fuerzas Virtual (VFF) 15
6.5 approachWall — Aproximación inicial 15
7. Controlador PID (PIDController.m) 16
7.1 Constructor 16
7.2 Métodos 16
7.3 Algoritmo 16

7.4 Ganancias por defecto del framework 16
8. Capa de Sensores y Odometría 17
8.1 readOdometry.m 17
8.2 readSensors.m 17
8.3 Mapa de montaje de sonares 17
8.4 angleDiff.m 18
9. Dashboard de Visualización (RobotDashboard.m) 19
9.1 Paneles del dashboard 19
9.2 Constructor y métodos 19
9.3 Buffer circular 19
10. Generación de Informes (ReportFactory.m) 20
10.1 Uso 20
10.2 Parámetros 20
10.3 Contenido de logData 21
11. Guía de Inicio Rápido 22
11.1 Primera misión: navegar a un punto 22
11.2 Misión con waypoints 22
11.3 Seguimiento de pared 23
12. Extensión del Framework: Nuevos Comportamientos 24
12.1 Paso 1: Crear la función de comportamiento 24
12.2 Paso 2: Añadir parámetros a mc.Config 24
12.3 Paso 3: Crear el método público en MissionControl 25
12.4 Paso 4: Usar el nuevo comportamiento 25
12.5 Buenas prácticas 26
13. Resolución de Problemas Frecuentes 27
14. Tabla de Referencia Rápida de la API 28

1. Introducción

El AmigoBot MATLAB Framework es un sistema de software modular diseñado para programar, controlar y monitorizar el robot diferencial AmigoBot utilizando MATLAB como entorno de desarrollo y ROS (Robot Operating System) como middleware de comunicaciones. El framework abstrae la complejidad de la interfaz ROS y ofrece una API de alto nivel orientada a misiones que permite al usuario definir comportamientos complejos de navegación autónoma con pocas líneas de código.

El diseño sigue una arquitectura en capas: la capa de infraestructura gestiona la conexión ROS y la lectura de sensores; la capa de comportamientos encapsula los algoritmos de control (PID go-to-point, seguimiento de pared, evitación de obstáculos); y la capa de misión, implementada en MissionControl, orquesta dichos comportamientos y registra los datos de ejecución para su posterior análisis.

1.1 Objetivo del documento

Esta documentación está dirigida a estudiantes y profesores con conocimientos básicos de ROS y MATLAB. Su propósito es servir de referencia exhaustiva para: entender la arquitectura del framework, configurar y lanzar misiones, ajustar parámetros de control, extender el sistema con nuevos comportamientos y generar informes de misión.

1.2 Requisitos del sistema

Componente	Requisito
MATLAB	R2020b o superior con Robotics System Toolbox
ROS	ROS 1 (Noetic o Melodic)
Robot	AmigoBot con sonar (8 canales) y LiDAR
Conexión de red	PC y robot en la misma subred IP
Sistema operativo	Windows, macOS o Linux (lado MATLAB)

2. Arquitectura del Framework

2.1 Estructura de ficheros

El proyecto se organiza en los siguientes ficheros dentro de la carpeta raíz robot_control/:

Fichero	Responsabilidad
`main_robot.m`	Punto de entrada. Inicializa ROS, crea MissionControl y define la misión.
`conect.m`	Establece la conexión ROS con el simulador (STDR).
`conect_irl.m`	Establece la conexión ROS con el robot físico.
`MissionControl.m`	API de alto nivel. Orquesta todos los comportamientos.
`RobotDashboard.m`	Visualización en tiempo real: mapa, radar de sensores y velocidades.
`PIDController.m`	Controlador PID discreto genérico con anti-windup.
`ReportFactory.m`	Generador de informes gráficos post-misión en modo oscuro.
`gotoPoint.m`	Comportamiento de navegación a punto con PID (modo point-shoot).
`driveToPoint.m`	Navegación a punto con PID desacoplado (modo continuous).
`wallFollower.m`	Seguimiento de pared mediante PID lateral.
`obstacleAvoidance.m`	Evitación reactiva de obstáculos mediante Campo de Fuerzas Virtual (VFF).
`approachWall.m`	Aproximación suave a la pared antes de iniciar el seguimiento.
`readOdometry.m`	Parsea el mensaje de odometría ROS a una estructura de pose.
`readSensors.m`	Agrega todos los sensores (sonar + LiDAR) en una estructura unificada.
`sendVelocity.m`	Publica un comando de velocidad (Twist) en ROS.
`angleDiff.m`	Diferencia angular normalizada a [-π, +π].

2.2 Diagrama de capas

El framework se estructura en cuatro capas funcionales bien diferenciadas:

CAPA 4 — USUARIO / MISIÓN
main_robot.m · Sección MISSION
CAPA 3 — ORQUESTACIÓN
MissionControl.m · RobotDashboard.m · ReportFactory.m
CAPA 2 — COMPORTAMIENTOS
gotoPoint · driveToPoint · wallFollower · obstacleAvoidance · approachWall · PIDController
CAPA 1 — INFRAESTRUCTURA ROS
conect.m / conect_irl.m · readOdometry · readSensors · sendVelocity · angleDiff

2.3 Flujo de ejecución

El ciclo de control principal opera a la frecuencia configurada (por defecto 10 Hz) y sigue siempre el mismo patrón por iteración:

1. **Lectura de odometría**: readOdometry() parsea el mensaje /odom y devuelve la pose actual (x, y, θ, vx, wz).
2. **Lectura de sensores**: readSensors() agrega los 8 sonares y el LiDAR en una única estructura.
3. **Selección de comportamiento**: MissionControl decide qué comportamiento está activo (gotoPoint, wallFollower, obstacleAvoidance).
4. **Cálculo de control**: El comportamiento activo calcula los comandos de velocidad (v, w).
5. **Publicación**: sendVelocity() publica el mensaje geometry_msgs/Twist en /cmd_vel.
6. **Registro y visualización**: Los datos se guardan en mission_log_ y se envían al RobotDashboard.
7. **Espera de temporización**: waitfor(rate) sincroniza el bucle a la frecuencia nominal.

3. Conexión con ROS

El framework proporciona dos scripts de inicialización diferenciados: uno para entorno de simulación (conect.m) y otro para el robot físico (conect_irl.m). Ambos establecen las mismas variables de entorno, inicializan el nodo MATLAB, se suscriben a los topics de sensores y crean el publicador de velocidad.

3.1 Conexión al simulador (conect.m)

Se utiliza cuando el robot corre en STDR. Configura las IPs del PC y del host del simulador y espera confirmación de odometría del robot con frame_id 'robot0'.

```
% Editar estas dos líneas según la red de laboratorio
PC_IP  = "192.168.100.158";   % IP del PC con MATLAB
IP_SIM = "192.168.100.166";   % IP de la máquina con STDR

setenv('ROS_MASTER_URI', 'http://' + IP_SIM + ':11311');
setenv('ROS_IP', PC_IP);
rosshutdown;
rosinit();
```

3.2 Conexión al robot físico (conect_irl.m)

Se usa en laboratorio con el AmigoBot real. Adicionalmente activa los motores enviando un mensaje Int32 con valor 1 al topic /cmd_motor_state, y espera odometría con frame_id 'base_link'.

```
% Editar estas dos líneas con las IPs del laboratorio
PC_IP  = "172.29.31.105";
IP_SIM = "172.29.30.179";    % IP del robot (ROS Master)

% Activar motores
pub_enable       = rospublisher('/cmd_motor_state', 'std_msgs/Int32');
msg_enable_motor = rosmessage(pub_enable);
msg_enable_motor.Data = 1;
send(pub_enable, msg_enable_motor);
```

NOTA main_robot.m detecta automáticamente si ROS ya está inicializado. Si el nodo ya está activo y los suscriptores existen, omite la conexión y continúa con la misión anterior, evitando reconexiones innecesarias.

3.3 Topics ROS utilizados

Topic	Tipo de mensaje	Descripción
`/robot0/odom` o `/pose`	nav_msgs/Odometry	Posición y velocidades del robot
`/robot0/sonar_N` (N=0…7)	sensor_msgs/Range	Distancias de los 8 sonares
`/robot0/laser_1` o `/scan`	sensor_msgs/LaserScan	Escáner láser 360°
`/robot0/cmd_vel` o `/cmd_vel`	geometry_msgs/Twist	Comandos de velocidad (publicador)
`/cmd_motor_state`	std_msgs/Int32	Activación de motores (solo IRL)

3.4 Estructura ros_handles

Una vez inicializado, todos los manejadores ROS se agrupan en la estructura ros_handles, que se pasa como argumento al constructor de MissionControl:

```
ros_handles.pub    = pub;          % rospublisher de /cmd_vel
ros_handles.msg    = msg_vel;      % mensaje Twist pre-reservado
ros_handles.odom   = odom;         % rossubscriber de /odom
ros_handles.sonars = {sonar0, sonar1, sonar2, sonar3,
                      sonar4, sonar5, sonar7, sonar6};
ros_handles.laser  = laser;        % rossubscriber del LiDAR
```

AVISO El orden del array sonars es relevante: los índices MATLAB 1-8 deben corresponder a los ángulos [+90°, +54°, +18°, -18°, -54°, -90°, +135°, -135°] respectivamente. Si se conectan en otro orden, las lecturas laterales del robot serán incorrectas.

4. MissionControl — API de Alto Nivel

MissionControl es la clase principal del framework. Es una clase handle de MATLAB, lo que significa que se trabaja siempre con la misma instancia por referencia. Orquesta todos los comportamientos de navegación, gestiona el registro de datos y comunica con el dashboard de visualización.

4.1 Constructor

```
mc = MissionControl(ros_handles);

% Con parámetros opcionales:
mc = MissionControl(ros_handles, ...
    'Rate',          10, ...   % Frecuencia del bucle de control [Hz]
    'ShowDashboard', true);    % Abrir ventana de visualización en tiempo real
```

4.2 Referencia completa de métodos públicos

mc.gotoPoint(target, ...)

Navega hacia un punto del espacio global. Admite orientación final opcional y dos modos de navegación.

```
mc.gotoPoint([x, y])                          % Navegar a posición
mc.gotoPoint([x, y, theta])                   % Navegar + orientación final
mc.gotoPoint([x, y], 'AvoidObstacles', true)  % Con evitación de obstáculos
mc.gotoPoint([x, y], 'Mode', 'continuous')    % Arco suave (defecto)
mc.gotoPoint([x, y], 'Mode', 'point-shoot')   % Parar, girar, avanzar
```

Parámetro	Descripción
`target`	[x, y] o [x, y, theta] en metros y radianes (marco global).
`AvoidObstacles`	Si true, activa obstacleAvoidance cuando el LiDAR detecta un obstáculo próximo. Por defecto false.
`Mode`	'continuous': PID angular y lineal independientes. Trayectoria suave en arco. 'point-shoot': Fase 1 alineación, Fase 2 avance. Más preciso, puede producir espirales amplias si el objetivo está detrás.

mc.drive(target_local, ...)

Versión de gotoPoint en coordenadas locales del robot. El eje X local apunta hacia delante, el eje Y local apunta a la izquierda.

```
mc.drive([1.0, 0.0])        % Avanzar 1 m en línea recta
mc.drive([0.0, 0.5])        % Desplazarse 0.5 m a la izquierda
mc.drive([1.0, 0.0, pi/4])  % Avanzar 1 m y quedar a 45° izquierda
```

NOTA drive() convierte las coordenadas locales al marco global aplicando la matriz de rotación con el yaw actual del robot, y delega internamente en gotoPoint(). Todos los parámetros opcionales de gotoPoint() son igualmente válidos.

mc.followWall(side, desired_dist, ...)

Inicia el comportamiento de seguimiento de pared. El robot mantiene una distancia lateral constante a la pared mientras navega.

```
mc.followWall('left',  0.4)                        % Seguir pared izquierda a 0.4 m
mc.followWall('right', 0.5, 'Duration', 30)        % Seguir pared derecha 30 segundos
mc.followWall('left',  0.4, 'UseApproach', true)   % Aproximarse antes de seguir
```

Parámetro	Descripción
`side`	'left' o 'right'. Indica qué pared lateral seguir.
`desired_dist`	Distancia objetivo a la pared en metros.
`Duration`	Tiempo máximo de seguimiento en segundos. Por defecto Inf (indefinido).
`UseApproach`	Si true, el robot busca primero la pared más cercana antes de seguirla. Por defecto false.

mc.addWaypoint(wp) y mc.runMission()

Permiten encolar una secuencia de puntos de paso y ejecutarlos en orden.

```
mc.addWaypoint([1.0, 0.0]);            % Encolar punto (x, y)
mc.addWaypoint([1.0, 2.0]);
mc.addWaypoint([0.0, 0.0, 0]);         % Encolar punto con orientación final

mc.runMission();                         % Ejecutar todos en orden
mc.runMission('AvoidObstacles', true, 'PauseBetween', 1.0);
```

Parámetro (runMission)	Descripción
`AvoidObstacles`	Activa la evitación de obstáculos en todos los waypoints. Por defecto true.
`PauseBetween`	Pausa en segundos entre waypoints consecutivos. Por defecto 0.5 s.

- mc.clearWaypoints() — Vacía la cola de waypoints.

mc.stop()

Publica velocidad cero en /cmd_vel y pone el estado interno a IDLE. Debe usarse siempre en el bloque catch de la misión para garantizar una parada de emergencia ante cualquier excepción.

mc.setVelocity(v, w)

Control manual directo. Publica el par (v, w) en /cmd_vel sin pasar por ningún comportamiento. Útil para pruebas puntuales o teleoperación desde MATLAB.

mc.setGains(nombre, [Kp Ki Kd])

Actualiza las ganancias de uno de los tres controladores PID internos y reinicia su estado (integral y derivativo).

```
mc.setGains('gotoPoint_angle', [2.0, 0.05, 0.30]);  % PID de ángulo (goto)
mc.setGains('gotoPoint_dist',  [0.5, 0.01, 0.10]);  % PID de distancia (goto)
mc.setGains('wallFollow',      [1.5, 0.02, 0.40]);  % PID lateral (wall)
```

mc.resetMissionLog() y mc.getMissionLog()

resetMissionLog() debe llamarse antes de la misión para inicializar el registro de datos. getMissionLog() devuelve la estructura con todos los datos acumulados para pasarla a ReportFactory.

```
mc.resetMissionLog();          % Inicializar antes de la misión

% ... ejecución de la misión ...

log = mc.getMissionLog();      % Obtener datos al finalizar
```

5. Configuración de Parámetros

Todos los parámetros de control se almacenan en la propiedad pública mc.Config. Esta estructura puede consultarse e incluso modificarse directamente en tiempo de ejecución, aunque se recomienda usar mc.setGains() para los parámetros PID, ya que esta función además reinicia el estado del controlador.

5.1 Parámetros de gotoPoint

Campo (mc.Config.gotoPoint)	Valor por defecto	Unidad	Descripción
`pid_angle`	[2.0, 0.05, 0.30]	—	Ganancias [Kp Ki Kd] del PID angular.
`pid_dist`	[0.5, 0.01, 0.10]	—	Ganancias [Kp Ki Kd] del PID de distancia.
`dt`	0.1	s	Período del bucle de control (1/Rate).
`dist_tol`	0.10	m	Radio de aceptación de posición.
`angle_tol`	0.06	rad	Tolerancia de orientación final (Fase 2).
`max_v`	0.70	m/s	Velocidad lineal máxima.
`max_w`	0.70	rad/s	Velocidad angular máxima.
`slow_dist`	0.50	m	Distancia a la que comienza la rampa de desaceleración.

5.2 Parámetros de wallFollow

Campo (mc.Config.wallFollow)	Valor por defecto	Unidad	Descripción
`pid`	[1.5, 0.02, 0.40]	—	Ganancias [Kp Ki Kd] del PID lateral.
`max_v`	0.70	m/s	Velocidad lineal máxima.
`max_w`	1.00	rad/s	Velocidad angular máxima.
`front_clear`	0.55	m	Distancia frontal mínima antes de girar en esquina.
`side / desired_dist`	se pasan en llamada	—	Lado ('left'/'right') y distancia objetivo. Se sobreescriben en cada llamada a followWall().

5.3 Parámetros de obstacleAvoid

Campo (mc.Config.obstacleAvoid)	Valor por defecto	Unidad	Descripción
`influence_radius`	1.20	m	Distancia a partir de la cual un obstáculo genera fuerza repulsiva.
`emergency_dist`	0.35	m	Distancia frontal que activa parada de emergencia y giro forzado.
`clear_dist`	0.60	m	Umbral para declarar el camino despejado (flag is_clear).
`max_v`	0.25	m/s	Velocidad lineal máxima durante evitación.
`max_w`	1.20	rad/s	Velocidad angular máxima durante evitación.

5.4 Modificación directa de Config

Cualquier campo puede modificarse directamente antes o durante la misión:

```
% Aumentar la distancia de aceptación de posición
mc.Config.gotoPoint.dist_tol = 0.20;

% Reducir la velocidad máxima en entorno estrecho
mc.Config.gotoPoint.max_v = 0.30;
mc.Config.wallFollow.max_v = 0.30;

% Hacer la evitación más agresiva
mc.Config.obstacleAvoid.influence_radius = 1.50;
mc.Config.obstacleAvoid.emergency_dist   = 0.50;
```

AVISO Modificar mc.Config.gotoPoint.pid_angle o mc.Config.gotoPoint.pid_dist directamente no reinicia el acumulador integral del controlador PID. Para una actualización limpia, usar siempre mc.setGains().

6. Comportamientos de Navegación

6.1 gotoPoint — Modo point-shoot

Implementado en gotoPoint.m. Utiliza un enfoque de dos fases secuenciales: primero alinea el robot con la dirección al objetivo (reduciendo la velocidad lineal proporcionalmente al desalineamiento), y una vez alcanzada la posición, ejecuta opcionalmente una rotación final hasta el ángulo deseado.

- **Fase 1 (navegación)**: El PID angular corrige el rumbo continuamente. La velocidad lineal escala con cos(error_angular) y con la rampa slow_dist. Esto penaliza el avance cuando el robot apunta en la dirección equivocada.
- **Fase 2 (orientación final)**: Solo activa si se especifica theta en el target. El robot se detiene y gira hasta alcanzar angle_tol.

NOTA Este modo puede producir espirales amplias si el objetivo está situado detrás del robot, ya que el robot intentará girar y avanzar simultáneamente.

6.2 driveToPoint — Modo continuous

Implementado en driveToPoint.m. El PID angular y el PID lineal operan completamente en paralelo y de forma independiente. No existe penalización del avance por desalineamiento, lo que produce trayectorias más suaves y orgánicas. Es el modo por defecto al llamar a mc.gotoPoint().

Este modo es especialmente adecuado cuando el objetivo es alcanzable sin obstáculos y se desea una trayectoria continua (por ejemplo, en misiones de waypoints encadenados).

Característica	Modo continuous	Modo point-shoot
Trayectoria	Arco suave	Girar → avanzar
PID lineal y angular	Independientes	Acoplados por cos(error)
Precisión posicional	Moderada	Alta
Objetivo detrás	Puede oscilar	Gira y avanza
Uso recomendado	Waypoints en ruta abierta	Posicionamiento preciso

6.3 wallFollower — Seguimiento de pared

Implementado en wallFollower.m. El robot mantiene una distancia lateral constante a la pared usando el sonar directamente lateral (sonar0 para izquierda, sonar5 para derecha) y el sonar diagonal frontal (sonar1 o sonar4) para anticipar esquinas.

- **Situación normal**: PID lateral sobre la mezcla ponderada (70% error lateral, 30% error diagonal). La velocidad lineal se reduce en proporción al error de distancia.
- **Obstáculo frontal**: Cuando la distancia frontal (LiDAR o media de sonares frontales) cae por debajo de front_clear, el robot gira alejándose de la pared con intensidad proporcional a la obstrucción. Esto permite negociar esquinas interiores y exteriores.

NOTA El sonar que devuelve NaN (sin eco válido) se sustituye por 3.0 m (valor de fallback). Esto evita que una lectura inválida produzca un error de control desproporcionado.

6.4 obstacleAvoidance — Campo de Fuerzas Virtual (VFF)

Implementado en obstacleAvoidance.m. Cada lectura de sonar y cada rayo LiDAR dentro del radio de influencia genera un vector de fuerza repulsiva en el marco del robot. La resultante de todas las fuerzas determina el rumbo de escape y la velocidad.

- **Pesos relativos**: Los sonares tienen peso completo; los rayos LiDAR dentro del sector frontal ±60° tienen peso 0.4 para evitar sobreponderar la nube densa de datos.
- **Parada de emergencia**: Si el LiDAR detecta un obstáculo a menos de emergency_dist en el sector frontal, la velocidad lineal se fija a 0 y se impone el máximo giro hacia el lado con más espacio libre.
- **is_clear**: Flag de retorno que indica si todos los sonares frontales (índices 2-5) y el LiDAR reportan distancias superiores a clear_dist. MissionControl lo usa para reactivar gotoPoint tras el esquive.

6.5 approachWall — Aproximación inicial

Implementado en approachWall.m. Se activa cuando UseApproach=true en followWall(). El robot se orienta hacia el punto más cercano detectado por el LiDAR, mezclando la dirección de avance y la alineación lateral en función de la distancia actual.

El parámetro alpha regula esta mezcla: vale 0 cuando el robot está lejos (avanza de frente) y 1 cuando está cerca de la distancia objetivo (se alinea lateralmente). La velocidad lineal se reduce a medida que alpha aumenta.

7. Controlador PID (PIDController.m)

PIDController es una clase handle genérica que implementa un controlador PID discreto de tiempo con anti-windup por clamping simétrico y diferencial por diferencia hacia atrás (backward difference). Es independiente del framework y puede usarse en cualquier proyecto MATLAB con ROS.

7.1 Constructor

```
pid = PIDController(Kp, Ki, Kd);

% Con parámetros opcionales:
pid = PIDController(Kp, Ki, Kd, ...
    'MaxOutput',    1.5,  ...   % Saturación superior de salida
    'MinOutput',   -1.5,  ...   % Saturación inferior de salida
    'MaxIntegral',  5.0);       % Anti-windup: límite del acumulador
```

7.2 Métodos

Método	Descripción
`output = pid.compute(error, dt)`	Calcula la salida PID para el ciclo actual. En la primera llamada, inicializa el estado derivativo para evitar pico transitorio.
`pid.reset()`	Reinicia el acumulador integral y el error previo. Esencial al cambiar de objetivo.
`pid.setGains(Kp, Ki, Kd)`	Actualiza las ganancias y llama a reset() automáticamente.

7.3 Algoritmo

La implementación sigue las ecuaciones estándar del PID discreto:

```
integral  = clamp(integral + error * dt, -MaxIntegral, +MaxIntegral)
derivative = (error - prev_error) / dt
raw_output = Kp * error + Ki * integral + Kd * derivative
output     = clamp(raw_output, MinOutput, MaxOutput)
```

NOTA El período dt tiene un suelo de 1e-6 s para evitar división por cero. En el primer ciclo, prev_error se inicializa con el error actual para que el término derivativo arranque en cero.

7.4 Ganancias por defecto del framework

Controlador	Kp	Ki	Kd	Uso
gotoPoint_angle	2.0	0.05	0.30	Corrección de rumbo
gotoPoint_dist	0.5	0.01	0.10	Velocidad lineal
wallFollow	1.5	0.02	0.40	Distancia lateral

8. Capa de Sensores y Odometría

8.1 readOdometry.m

Parsea el último mensaje nav_msgs/Odometry del suscriptor y devuelve una estructura de pose estandarizada:

```
pose = readOdometry(odom_sub);

% Campos de retorno:
% pose.x       — Posición X en el marco global [m]
% pose.y       — Posición Y en el marco global [m]
% pose.theta   — Yaw (heading) en el marco global [rad]
% pose.vx      — Velocidad lineal hacia adelante [m/s]
% pose.wz      — Velocidad angular [rad/s]
% pose.stamp   — Timestamp del mensaje [s]
```

NOTA El yaw se extrae del cuaternion de orientación mediante la fórmula atan2(2(WZ+XY), 1-2(Y^2+Z^2)), estándar para rotaciones ZYX.

8.2 readSensors.m

Agrega los 8 sonares y el LiDAR en una única estructura sensors, que es el formato estándar esperado por todos los comportamientos:

```
sensors = readSensors(sonar_subs, laser_sub);

% Campos de retorno:
% sensors.sonar            — [1x8] Distancias sonar [m], NaN si inválido
% sensors.sonar_angles     — [1x8] Ángulos de montaje [rad]
% sensors.laser_ranges     — [1xN] Distancias LiDAR [m], NaN si inválido
% sensors.laser_angles     — [1xN] Ángulo de cada rayo [rad]
% sensors.laser_min_front  — Mínimo en sector frontal ±45° [m]
```

8.3 Mapa de montaje de sonares

Los 8 sonares del AmigoBot cubren un arco frontal completo más dos diagonales traseras. Los ángulos siguen la convención ROS (0° = adelante, positivo = izquierda/CCW):

Sonar 0	Sonar 1	Sonar 2	Sonar 3	Sonar 4	Sonar 5	Sonar 6	Sonar 7
+90°	+54°	+18°	−18°	−54°	−90°	+135°	−135°
Izq.	Fr-Izq.	Fr-Izq.	Fr-Der.	Fr-Der.	Der.	Tr-Izq.	Tr-Der.

NOTA Las lecturas sonar que devuelvan un valor fuera de [MinRange, MaxRange] o superior a 3.5 m son descartadas y registradas como NaN. Esto evita que ecos erróneos afecten al control.

8.4 angleDiff.m

Función utilitaria que calcula la diferencia angular con manejo correcto del salto en ±π:

```
diff = angleDiff(a, b)    % Devuelve (a - b) normalizado a [-pi, +pi]

% Ejemplo:
% angleDiff(pi-0.1, -pi+0.1) = -0.2    (camino corto por ±pi)
```

9. Dashboard de Visualización (RobotDashboard.m)

RobotDashboard es una clase handle que abre una ventana MATLAB de 1400×800 píxeles con tres paneles de visualización actualizados en tiempo real durante la ejecución de la misión. El dashboard usa un buffer circular pre-asignado para mantener el rendimiento incluso en sesiones largas.

9.1 Paneles del dashboard

- **Panel izquierdo — Mapa de trayectoria**: Muestra la trayectoria recorrida (línea azul), la posición actual del robot (punto amarillo), la dirección de avance (flecha blanca), los puntos LiDAR proyectados en el marco global (nube verde) y el marcador del objetivo (estrella roja).
- **Panel superior derecho — Radar de sensores locales**: Vista en el marco del robot. Muestra los 8 haces de sonar como líneas coloreadas de verde (lejos) a rojo (cerca), y la nube LiDAR en coordenadas locales. Las distancias se indican con círculos de referencia cada 1 m.
- **Panel inferior derecho — Perfil de velocidades**: Gráfica temporal con doble eje Y. Eje izquierdo (verde/cian): velocidad lineal comandada (discontinua) y real (sólida). Eje derecho (naranja): velocidad angular comandada y real. Ventana deslizante de 30 s.
- **Barra de estado inferior**: Muestra en tiempo real: comportamiento activo, pose (x, y, θ), y velocidades comandadas y reales.

9.2 Constructor y métodos

```
db = RobotDashboard();
db = RobotDashboard('MaxHistory', 800);  % Tamaño del buffer circular

db.update(pose, sensors, v_cmd, w_cmd, v_real, w_real, behavior);
db.setTarget([x, y]);      % Actualizar marcador de objetivo
db.setTarget([]);          % Ocultar marcador
db.close();                % Cerrar ventana
```

NOTA MissionControl llama a db.update() automáticamente en cada iteración del bucle de control. El usuario no necesita gestionar el dashboard directamente; accede a él a través de mc.Dashboard si necesita controlarlo manualmente.

9.3 Buffer circular

Los arrays de historial (posición, tiempos, velocidades) están pre-asignados en la construcción del objeto con MaxHistory elementos. Cuando se supera la capacidad, el índice de escritura da la vuelta al principio (buffer circular), evitando la costosa reasignación dinámica de memoria que degradaría el rendimiento en tiempo real.

10. Generación de Informes (ReportFactory.m)

ReportFactory es una clase de métodos estáticos que genera una figura MATLAB de dos paneles en modo oscuro a partir de los datos registrados en mission_log_. La figura se guarda automáticamente como .fig y .png en la carpeta indicada.

10.1 Uso

```
% Firma completa:
ReportFactory.generateReport(logData, target_info, mission_name, save_folder)

% Ejemplo típico:
ReportFactory.generateReport(mc.getMissionLog(), [2.0, 2.0], 'Mision1', 'reportes');

% Con opciones:
ReportFactory.generateReport(mc.getMissionLog(), [], 'WallFollow_Test',
'reportes', ...
    'ShowLidar',       true,  ...  % Mostrar mapa de paredes filtrado
    'ShowOrientation', true,  ...  % Mostrar flechas de yaw
    'ShowGains',       true,  ...  % Mostrar caja de parámetros PID
    'ShowTarget',      false);     % Ocultar marcador de destino
```

10.2 Parámetros

Argumento / Opción	Tipo	Descripción
`logData`	struct	Estructura devuelta por mc.getMissionLog().
`target_info`	[x, y] o []	Posición del objetivo para el marcador. Pasar [] si no aplica (e.g. wall following).
`mission_name`	string	Nombre de la misión. Se usa como título y nombre de archivo.
`save_folder`	string	Carpeta de destino. Se crea si no existe.
`ShowLidar`	logical (true)	Muestra un mapa de paredes filtrado por densidad (≥11 puntos por celda de 10 cm).
`ShowOrientation`	logical (true)	Superpone flechas de orientación (quiver) cada N muestras.
`ShowGains`	logical (true)	Muestra una caja de texto con las ganancias PID y estadísticas de la misión.
`ShowTarget`	logical (true)	Muestra el marcador de destino (estrella roja) si target_info no está vacío.

10.3 Contenido de logData

La estructura mission_log_ contiene los siguientes campos relevantes para el informe:

Campo	Contenido
`t`	Vector de tiempos relativos [s] desde el inicio de la misión.
`x, y, theta`	Vectores de posición [m] y orientación [rad] a cada instante.
`v_cmd, w_cmd`	Velocidades lineal y angular comandadas por el controlador.
`v_real, w_real`	Velocidades reales leídas de la odometría (pose.vx, pose.wz).
`wall_points_x/y`	Puntos LiDAR proyectados en el marco global (decimados cada 10 ciclos, 1 de cada 10 rayos).
`pid_angle, pid_dist`	Referencias a los objetos PIDController en el momento del registro.

11. Guía de Inicio Rápido

11.1 Primera misión: navegar a un punto

Sigue estos pasos para ejecutar una misión básica de go-to-point con el robot físico:

8. Abrir MATLAB en la carpeta robot_control/ (o añadirla al path).
9. Editar las IPs en conect_irl.m según la red del laboratorio.
10. En main_robot.m, localizar la sección MAIN MISSION y escribir la misión deseada.
11. Ejecutar main_robot en MATLAB: >> main_robot
12. Observar el dashboard en tiempo real y los mensajes de la consola MATLAB.
13. Al finalizar, revisar el informe generado en la carpeta 'reportes/'.

```
% Contenido de la sección MAIN MISSION para un punto simple:
mc.resetMissionLog();

try
    mc.gotoPoint([2.0, 1.5]);
catch exception
    fprintf(2, 'Error: %s\n', exception.message);
    mc.stop();
end

ReportFactory.generateReport(mc.getMissionLog(), [2.0, 1.5], 'Test1', 'reportes');
```

11.2 Misión con waypoints

```
mc.resetMissionLog();

mc.addWaypoint([1.0, 0.0]);
mc.addWaypoint([1.0, 1.0]);
mc.addWaypoint([0.0, 1.0]);
mc.addWaypoint([0.0, 0.0, 0]);  % Volver al origen mirando al frente

try
    mc.runMission('AvoidObstacles', true);
catch e
    mc.stop();
end
```

11.3 Seguimiento de pared

```
mc.setGains('wallFollow', [0.9, 0.008, 0.75]);  % Afinar ganancias
mc.resetMissionLog();

try
    mc.followWall('left', 0.4, 'Duration', 60);
catch e
    mc.stop();
end

ReportFactory.generateReport(mc.getMissionLog(), [], 'WallFollow', 'reportes');
```

12. Extensión del Framework: Nuevos Comportamientos

El framework está diseñado para ser fácilmente ampliable. Un nuevo comportamiento es una función MATLAB estándar (.m) que recibe la pose y/o los sensores del robot y devuelve comandos de velocidad (v, w). Para integrarlo completamente en el sistema, se siguen cuatro pasos.

12.1 Paso 1: Crear la función de comportamiento

La función debe seguir la firma estándar. Como mínimo, recibe la estructura de sensores o de pose y devuelve (v, w):

```
function [v, w] = miComportamiento(pose, sensors, config)
%MICOMPORTAMIENTO Descripción breve del comportamiento.
%
%  INPUTS
%    pose    — Struct de readOdometry() con campos .x .y .theta .vx .wz
%    sensors — Struct de readSensors() con sonares y LiDAR
%    config  — Struct de parámetros (pasado desde mc.Config)
%
%  OUTPUTS
%    v  — Velocidad lineal [m/s]
%    w  — Velocidad angular [rad/s]

    % --- Implementación del comportamiento ---
    v = 0.2;
    w = 0.0;
end
```

12.2 Paso 2: Añadir parámetros a mc.Config

Si el comportamiento necesita parámetros propios, añadirlos al método estático defaultConfig_() de MissionControl.m:

```
% Dentro de MissionControl.defaultConfig_():
cfg.miComportamiento.max_v     = 0.40;   % [m/s]
cfg.miComportamiento.umbral    = 0.80;   % [m]
cfg.miComportamiento.kp        = 1.20;
```

12.3 Paso 3: Crear el método público en MissionControl

Añadir un método público en MissionControl.m que gestione el bucle de control e invoque la función del paso 1:

```
function ejecutarMiComportamiento(obj, varargin)
    p = inputParser();
    addParameter(p, 'Duration', Inf, @isnumeric);
    parse(p, varargin{:});

    cfg     = obj.Config.miComportamiento;
    t_start = tic;
    while toc(t_start) < p.Results.Duration
        pose    = readOdometry(obj.ros_.odom);
        sensors = readSensors(obj.ros_.sonars, obj.ros_.laser);

        obj.behavior_ = 'MI_COMPORTAMIENTO';
        [v, w] = miComportamiento(pose, sensors, cfg);

        sendVelocity(obj.ros_.pub, obj.ros_.msg, v, w);
        obj.tickDashboard_(pose, sensors, v, w);
        obj.updateLog_(pose, sensors, v, w);
        waitfor(obj.rate_);
    end
    obj.stop();
end
```

12.4 Paso 4: Usar el nuevo comportamiento

Una vez añadido a MissionControl, el nuevo comportamiento se invoca exactamente igual que los existentes:

```
mc.resetMissionLog();
try
    mc.ejecutarMiComportamiento('Duration', 30);
catch e
    mc.stop();
end
```

NOTA El nombre del comportamiento que se pasa a tickDashboard_ (tercer argumento de obj.behavior_) aparecerá en la barra de estado del dashboard y en los logs de la consola. Usar nombres descriptivos en mayúsculas facilita la depuración.

12.5 Buenas prácticas

- **Saturación de salidas**: Aplicar siempre max/min sobre v y w antes de retornar para respetar los límites físicos del robot.
- **Manejo de NaN**: Comprobar siempre si las lecturas de sensores son NaN antes de usarlas en cálculos, especialmente en el LiDAR.
- **Reinicio de PID**: Si el comportamiento utiliza un PIDController, llamar a pid.reset() al inicio y cada vez que cambie el objetivo.
- **Documentación**: Incluir la cabecera de ayuda estándar MATLAB (%FUNCION descripción, INPUTS, OUTPUTS) para que help miComportamiento funcione correctamente.

13. Resolución de Problemas Frecuentes

Síntoma	Causa probable y solución
Timeout al esperar odometría	El ROS_MASTER_URI o ROS_IP son incorrectos. Verificar IPs con ping y que roscore esté activo en el robot.
El robot no se mueve	Los motores no están activados (solo IRL). Comprobar que conect_irl.m envió msg_enable_motor.Data = 1 correctamente.
Gotopoint produce espirales	El objetivo está detrás del robot y se usa modo continuous. Cambiar a Mode 'point-shoot' o reorientar el robot manualmente.
Wall follower oscila fuerte	Kd demasiado alta o Kp demasiado alta. Reducir Kp a ~0.7 y Kd a ~0.3 con mc.setGains.
Dashboard se queda congelado	MATLAB está ocupado en el bucle de control. Es normal; drawnow limitrate lo actualiza cuando puede. Si persiste, reducir MaxHistory.
ReportFactory genera figura en blanco	mc.resetMissionLog() no fue llamado antes de la misión. El campo t estará vacío.
Error 'no odometry from robot0'	El frame_id del mensaje de odometría no coincide. conect.m espera 'robot0'; conect_irl.m espera 'base_link'.
Lecturas sonar siempre NaN	El orden del array sonars en ros_handles no coincide con el montaje físico, o los topics tienen nombres distintos.
mc.stop() no detiene el robot	El publicador pub ha perdido la conexión. Reconectar con conect_irl.m y recrear mc.

14. Tabla de Referencia Rápida de la API

Llamada	Descripción
`MissionControl(handles, 'Rate', 10)`	Crear instancia del controlador de misión.
`mc.gotoPoint([x, y])`	Navegar a punto (x, y) en modo continuous.
`mc.gotoPoint([x, y, θ])`	Navegar a punto y alinear a orientación θ.
`mc.gotoPoint([x,y], 'AvoidObstacles', true)`	Navegar con evitación de obstáculos activa.
`mc.gotoPoint([x,y], 'Mode', 'point-shoot')`	Navegar en modo parar-girar-avanzar.
`mc.drive([xf, yl])`	Navegar en coordenadas locales del robot.
`mc.addWaypoint([x, y])`	Encolar un waypoint en la misión.
`mc.clearWaypoints()`	Vaciar la cola de waypoints.
`mc.runMission()`	Ejecutar todos los waypoints en cola.
`mc.followWall('left', 0.4)`	Seguir pared izquierda a 0.4 m.
`mc.followWall(..., 'Duration', 30)`	Seguir pared durante 30 segundos.
`mc.followWall(..., 'UseApproach', true)`	Aproximarse a la pared antes de seguirla.
`mc.stop()`	Parada de emergencia (v=0, w=0).
`mc.setVelocity(v, w)`	Comando manual directo de velocidad.
`mc.setGains('gotoPoint_angle', [Kp Ki Kd])`	Actualizar ganancias PID de rumbo.
`mc.setGains('gotoPoint_dist', [Kp Ki Kd])`	Actualizar ganancias PID de distancia.
`mc.setGains('wallFollow', [Kp Ki Kd])`	Actualizar ganancias PID de seguimiento de pared.
`mc.Config`	Estructura editable con todos los parámetros.
`mc.resetMissionLog()`	Inicializar el registro de datos de misión.
`mc.getMissionLog()`	Obtener la estructura de datos registrados.
`ReportFactory.generateReport(log, tgt, nombre, dir)`	Generar informe gráfico post-misión.